

LINQ Max Method in C# with Examples

In this article, I will discuss the **LINQ Max Method in C#** with Examples. Please read our previous article discussing the [LINQ Sum Aggregate Method in C#](#) with Examples. The LINQ Max() Method belongs to the category of Aggregate Operators. As part of this article, we will discuss the following pointers.

1. **What is the LINQ Max Method in C#?**
2. **Multiple Examples to Understand Sum Method using both Method and Query Syntax.**
3. **When should you use the LINQ Max Method in C#?**

What is the LINQ Max Method?

The LINQ Max method is used to find the maximum value in a collection. It's part of the **System.Linq** namespace can be used with arrays, lists, or any type that implements **IEnumerable<T>**. The Max method comes in various forms, allowing you to find the maximum value in different ways depending on the type of the collection and the data it contains.

Example to Understand LINQ Max Method in C#:

Let us understand the LINQ Max() method with Examples using C#. The following example returns the largest number from the collection using query and method syntax. The point that you need to remember is that the Max Method will work with numeric values only. In this case, the collection stores integer values, so we can apply the LINQ Max Method. We don't have any operator called max in the Query Syntax. So here, we need to use mixed syntax.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 10, 80, 50, 90, 60, 30, 70, 40, 20,
            //Using Method Syntax
            int MSLargestNumber = intNumbers.Max();
            //Using Query Syntax
            int QSLargestNumber = (from num in intNumbers
                                   select num).Max();
            Console.WriteLine("Largest Number = " + MSLargestNumber);
            Console.ReadKey();
        }
    }
}
```

Output: **Largest Number = 100**

Example to Understand LINQ Max Method with Where Extension Method using C#

Let us see an example to Understand how we can use the LINQ Max Method and the Where Extension Method in C# using both Method and Query Syntax. Now, our requirement is to return the largest number from the collection where the number is less than 50. The following example code exactly does the same.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 10, 80, 50, 90, 60, 30, 70, 40, 20,
            //Using Method Syntax
            int MSLargestNumber = intNumbers.Where(num => num < 50).Max();
            //Using Query Syntax
            int QSLargestNumber = (from num in intNumbers
                                   where num < 50
                                   select num).Max();
            Console.WriteLine("Largest Number = " + MSLargestNumber);
            Console.ReadKey();
        }
    }
}
```

Output: **Largest Number = 40**

Example to Understand How to use LINQ Max Method with Predicate in C#

Let us see an example of using the LINQ Max Method with Predicate in C# using both Method and Query Syntax. Instead of using the Where Extension method to filter the data, we can also use the other overloaded version of the Max method, which takes a Predicate as a parameter, and using the predicate, we can write the logic to filter the data. In the example below, within the Max Method, we use a Predicate and provide the condition whether the number is less than 50. If the number is less than 50, then we return that number; else, we return 0. The following example will return the largest number, less than 50.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 10, 80, 50, 90, 60, 30, 70, 40, 20,
            //Using Method Syntax
            int MSLargestNumber = intNumbers.Max(num => {
                if (num < 50)
                    return num;
                else
                    return 0;
            });
            Console.WriteLine("Largest Number = " + MSLargestNumber);
            Console.ReadKey();
        }
    }
}
```

```

        return 0;
    });

    Console.WriteLine("Largest Number = " + MSLargestNumber);
    Console.ReadKey();
}
}
}

```

Output: **Largest Number = 40**

Example to Understand LINQ Max Method with Complex Type in C#:

Let us see an example of using the LINQ Max Method with Complex Data Type in C# using both Method and Query Syntax. We are going to work with the following Employee class. As you can see, it is a very simple class with four properties: **ID**, **Name**, **Salary**, and **Department**. Here, we also created one method, i.e., **GetAllEmployees()**, which will return the list of all the employees and be our data source.

```

using System.Collections.Generic;
namespace LINQDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Department { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            List<Employee> listStudents = new List<Employee>()
            {
                new Employee{ID= 101,Name = "Preety", Salary = 10000, Department = "Sales"},
                new Employee{ID= 102,Name = "Priyanka", Salary = 15000, Department = "Sales"},
                new Employee{ID= 103,Name = "James", Salary = 50000, Department = "IT"},
                new Employee{ID= 104,Name = "Hina", Salary = 20000, Department = "Sales"},
                new Employee{ID= 105,Name = "Anurag", Salary = 30000, Department = "IT"},
                new Employee{ID= 106,Name = "Sara", Salary = 25000, Department = "Sales"},
                new Employee{ID= 107,Name = "Pranaya", Salary = 35000, Department = "IT"},
                new Employee{ID= 108,Name = "Manoj", Salary = 11000, Department = "Sales"},
                new Employee{ID= 109,Name = "Sam", Salary = 45000, Department = "IT"},
                new Employee{ID= 110,Name = "Saurav", Salary = 25000, Department = "Sales"}
            };
            return listStudents;
        }
    }
}

```

```
}
```

Now, our requirement is to return the highest salary among all the employees. The following example returns the highest salary among all the employees using the LINQ Max method with both Method and Query Syntax. Here, we specify the numeric Salary column using a lambda expression to the Max method.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSHighestSalary = Employee.GetAllEmployees()
                .Max(emp => emp.Salary);
            //Using Query Syntax
            var QSHighestSalary = (from emp in Employee.GetAllEmployees()
                                   select emp).Max(e => e.Salary);
            Console.WriteLine("Highest Salary = " + QSHighestSalary);
            Console.ReadKey();
        }
    }
}
```

Output: **Highest Salary = 50000**

Example using Max and Where Extension Method in C#:

Let us see an example of using both LINQ Max and Where Extension Methods with Complex Type using Method and Query Syntax. Our requirement is to return the Highest Salary in the IT department. The following example exactly does the same. Using the Where Extension Method, we are filtering the IT department employees, and using the Max method, we are specifying the Salary numeric column, which will return the Highest Salary of the IT department.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSHighestSalary = Employee.GetAllEmployees()
                .Where(emp => emp.Department == "IT")
                .Max(emp => emp.Salary);
            //Using Query Syntax
            var QSHighestSalary = (from emp in Employee.GetAllEmployees()
                                   where emp.Department == "IT"
                                   select emp).Max(e => e.Salary);
        }
    }
}
```

```

        where emp.Department == "IT"
        select emp).Max(e => e.Salary);
    Console.WriteLine("It Department Highest Salary = " + QSHighestSalary);
    Console.ReadKey();
}
}
}

```

Output: It Department Highest Salary = 35000

Let's rewrite the previous example using the custom predicate. As you can see, within the Max method, we are checking the Department property value, and if the Department is IT, we are returning the corresponding Salary. Else, we are returning 0.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSHighestSalary = Employee.GetAllEmployees()
                .Max(emp => {
                    if (emp.Department == "IT")
                        return emp.Salary;
                    else
                        return 0;
                });

            Console.WriteLine("It Department Highest Salary = " + MSHighestSalary);
            Console.ReadKey();
        }
    }
}

```

Output: It Department Highest Salary = 35000

Let's rewrite the previous example using the custom predicate. As you can see, within the Max method, we are checking the Department property value, and if the Department value is It, we are returning the corresponding Salary. Else, we are returning null.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)

```

```

    {
        //Using Method Syntax and Predicate
        var HighestSalaryMS = Employee.GetAllEmployees()
            .Max(emp => {
                if (emp.Department == "IT")
                    return emp.Salary;
                else
                    return null;
            });
        Console.WriteLine("IT Department Highest Salary = " + HighestSalaryMS);
        Console.ReadKey();
    }
}

```

Output: **IT Department Highest Salary = 35000**

Handling Nullable Types

The Max method also has overloads for handling nullable numeric types (int?, double?, etc.). The max method will ignore null values. For a better understanding, please have a look at the below example.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int?[] nullableNumbers = { 1, 2, 10, null, 4, 5 };
            int? max = nullableNumbers.Max();
            Console.WriteLine("Max = " + max);
            Console.ReadKey();
        }
    }
}

```

Output: **Max = 10**

When to Use the LINQ Max Method?

The Max method in LINQ is used when finding the maximum value in a collection. Here are some common scenarios when you might use the Max method:

- **Finding the Highest Value:** When you have a collection of numeric types (like int, double, and decimal), you want to find the highest number in that collection. This is useful in scenarios like determining the highest score in a game, the maximum temperature in a weather dataset, or the highest price in a list of products.

- **Maximum of Custom Objects:** If you have a collection of custom objects, you can use Max with a lambda expression to find the object with the highest value of a specific property. For example, finding the employee with the highest salary in a list of employees.
- **Maximum with Nullable Types:** LINQ's Max method can handle nullable types (int?, double?, etc.) and will return null if the collection is empty or consists only of null values.
- **Querying Databases:** When querying databases using LINQ to Entities or LINQ to SQL, you can use the Max method to retrieve the maximum value from a column in a database table.
- **Aggregation in Complex Queries:** In more complex LINQ queries, Max can be used as part of aggregation operations to find the maximum value in groups after applying GroupBy or other operations.

In the next article, I will discuss the [LINQ Min Aggregate Method in C#](#) with Examples. I hope you will understand the use and need of the Linq Max Aggregate Method in C# with Examples.